

SC102 L05: Unity 物理系统详解——详细学习笔记

涵盖：物理引擎概论 | 离散碰撞检测 (DCD) | 连续碰撞检测 (CCD) | 连接关节 | 2D 作用器 | BoxCast/SphereCast | 射线检测可视化

1 Unity 中的物理引擎

[p.4]

物理引擎是一系列模拟真实世界物理现象的算法。常见的模拟类型包括刚体物理 (Rigidbody)、软体物理、流体物理等。Unity 本身主要支持**刚体物理 (Rigidbody Dynamics)**——”实心”的、不可变形的物体之间的物理模拟，由 Rigidbody 组件实现。

Unity 内置的物理引擎分为两个独立部分：

- **3D 物理**：使用 NVIDIA 开发的 **PhysX** 引擎
- **2D 物理**：使用开源引擎 **Box2D**

两个引擎在 Unity 中共存，但不能同时作用在同一个物体上（2D 和 3D 物理是隔离的）。

刚体物理引擎主要做两件事：

1. **碰撞检测 (Collision Detection)**：判断物体之间是否发生接触或重叠
2. **刚体动力学模拟 (Rigidbody Dynamics Simulation)**：根据力、速度、质量等计算物体应该如何运动/反弹

2 物理引擎的基本结构

[p.5]

物理引擎以**固定的时间步长 (Fixed Timestep)**对物理组件进行更新模拟——这就是 `FixedUpdate()` 的作用。默认固定步长为 0.02 秒 (50Hz)。

关键理解：物理更新的时间步长和渲染帧的时间不是一回事。一帧画面渲染期间可能有 0 次、1 次或多次物理更新。

在 Unity 中，每次物理更新发生：

1. 调用所有 `FixedUpdate()`，更新物理组件行为

2. 唤醒所有 `yield return new WaitForFixedUpdate()` 的协程
3. 执行物理事件回调: `OnTriggerEnter/Stay/Exit`、`OnCollisionEnter/Stay/Exit`

物理插值 (Interpolation): 当两次 `FixedUpdate` 之间的间隔小于固定步长时, 后续的 `FixedUpdate` 会被跳过。高帧率下这可能导致卡顿/不协调。Unity 在渲染帧之间对物体位置进行插值以维持平滑过渡。Rigidbody 组件的 `Interpolation` 选项定义了这个行为:

- **None:** 不插值
- **Interpolate:** 基于上一帧位置插值 (适合被物理驱动的对象)
- **Extrapolate:** 基于当前速度外推 (适合玩家控制的物体)

3 离散碰撞检测 (Discrete Collision Detection, DCD)

[p.6]

DCD 是最基本的碰撞检测方式, 在离散的时间点进行物理计算。

每个物理帧中, DCD 的处理流程:

1. 获取物体上一帧的位置和速度, 计算出新位置和速度
2. 检测新位置是否与其他物体重叠 (穿透检测/Penetration Test)
3. 如果有重叠, 进入碰撞物理计算 (分离、计算反弹速度等), 得出最终位置和速度

DCD 的致命缺点: 快速移动的物体会在两帧之间“穿过”其他物体, 从而逃过碰撞检测——这就是经典的穿模/隧道效应 (Tunneling) 问题。

4 连续碰撞检测 (Continuous Collision Detection, CCD)

[p.7]

CCD 专门解决 DCD 的隧道效应问题。当物体速度很快时, 仅仅检测离散时间点的位置是不够的, 需要检测物体在整个运动路径上是否会与什么发生碰撞。

最自然的想法: 减少两次更新的时间间隔。但这会极大增加性能开销 (需要在更短的时间步长内做更多物理计算)。

4.1 Sweep-based CCD (扫掠式 CCD)

[p.8-9]

Sweep-based CCD 将物体的运动轨迹想象成一个“扫掠体积 (Swept Volume)”, 并使用这个较大的扫掠范围作为碰撞检测的形状。

算法流程:

1. 记录物体上一帧与当前帧的位置和线速度
2. 根据两帧的数据计算出一个移动范围（扫掠形状）
3. 对该移动范围内的物体进行碰撞检测
4. 如果碰撞了，计算碰撞响应并调整位置/速度

伪代码：

```
initial_pos = object.position
final_pos = object.position + object.velocity * time_step
swept_shape = CreateSweptShape(initial_pos, final_pos, object.radius)
if CheckCollision(swept_shape, other_swept_shape):
    HandleCollision(object, other)
else:
    object.position = final_pos
```

限制：

- 只依赖线速度，不能处理角速度（旋转）很大时的碰撞
- 性能消耗极高，尤其是扫掠范围内物体较多时

在 Unity 中设置 Rigidbody 的 Collision Detection 为 Continuous 或 Continuous Dynamic 来开启 Sweep-based CCD。

4.2 Speculative CCD（预测式 CCD）

[p.10-12]

Speculative CCD 根据上一帧数据预测物体当前帧的位置和速度，通过扩大物体的 AABB（轴对齐包围盒）来实现。

AABB (Axis-Aligned Bounding Box)：能完全包围物体、且各边与坐标轴平行的最小长方体。

算法原理：

- 根据预测的轨迹扩大物体的 AABB
- 物理引擎使用扩大的 AABB 进行碰撞检测
- 检测到碰撞后，考虑碰撞法线重新规划物体的位置

优势：能考虑角速度，对快速旋转物体的处理比 Sweep-based 更好。性能也比 Sweep-based 好一些。

限制：

- **Ghost Collision (幽灵碰撞)**: 当地面由多个物体拼接而成时, 引擎可能错误推断出不应该存在的碰撞点, 导致物体看起来”突然弹起来”
- **穿模问题**: AABB 只是预测范围, AABB 之外的物体可能被忽略而导致穿透

在 Unity 中设置 Collision Detection 为 Continuous Speculative 来开启。

4.3 三种碰撞检测对比

[p.13]

检测算法	精确度	性能消耗	Rigidbody 选项
离散碰撞检测 DCD	低	低	Discrete
Sweep-based CCD	高	高	Continuous / Continuous Dynamic
Speculative CCD	较高	较高	Continuous Speculative

实践建议:

- 静态/慢速物体: DCD 即可, 性能最优
- 高速子弹/投射物: Sweep-based CCD 或 Speculative CCD
- 玩家角色/需要精确碰撞: Speculative CCD (兼顾性能和旋转)

5 连接关节组件 (Joint Components)

5.1 Fixed Joint (固定关节)

[p.15]

Fixed Joint 将两个刚体”粘”在一起。类似于《塞尔达传说: 王国之泪》中的”究极手”技能。

重要属性:

- **Connected Body**: 连接的目标刚体组件
- **Break Force**: 断裂所需的力 (超过此力自动断开)
- **Break Torque**: 断裂所需的扭矩

应用场景: 将物体粘在一起构成复杂结构, 受足够大力时自动分离 (破坏效果)。

5.2 Hinge Joint (铰链关节)

[p.16]

Hinge Joint 为刚体添加”铰链连接”行为——两个物体围绕一个轴相对旋转。最常见的例子是门。

重要属性:

- **Connected Body:** 连接的目标刚体
- **Axis:** 铰链围绕的旋转轴
- **Anchor:** 铰链轴上的位置
- **Break Force / Break Torque:** 断裂条件

应用场景: 门、摆锤、钟摆、链条、风车叶片等一切围绕固定轴旋转的物体。

5.3 Spring Joint (弹簧关节)

[p.17]

Spring Joint 为刚体添加” 弹簧连接” 行为——有一股力不断尝试将两个刚体拉向同一个点，距离越远力越大（胡克定律思想）。

重要属性:

- **Connected Body:** 连接的目标刚体
- **Anchor:** 弹簧固定的位置
- **Spring:** 弹簧强度（弹性系数）
- **Damper:** 阻尼（减速快慢，防止无限振荡）
- **Break Force / Break Torque:** 断裂条件

6 2D 作用器 (Effector2D)

[p.18]

2D 作用器为 2D 物理物体添加更复杂的行为，**必须搭配 Collider2D 组件使用**（需要设置 Collider2D 的 `usedByEffector` 为 `true`）。

- **AreaEffector2D:** 在区域内对所有物体施加一个力（风力区域）
- **BuoyancyEffector2D:** 在区域内产生浮力（水面效果）
- **PlatformEffector2D:** 制作单向平台（能从下方跳上去，站在上面不会掉下来）
- **PointEffector2D:** 在一个点向四周施加吸引力或斥力（磁铁效果、黑洞效果）
- **SurfaceEffector2D:** 让接触表面的物体沿表面移动（传送带）

7 高级射线检测

7.1 基本 Raycast 的局限性

[p.20]

在 SC-101 中我们学习了 `Physics.Raycast`。但简单的单条射线在实际场景中有很多不足：

场景：2D 游戏中一个 3×3 的方块角色需要判断是否在地面上。

- 如果射线起点在角色中心：站在平台边缘时检测不到地面（中心已超出平台）
- 如果站在斜坡上：每个点到地面的距离不同，该用多长的射线？
- 如果射线无限长：跳起来时也会检测到地面

解决方案：使用形状射线检测（**Shape Cast**）而非单条线。

7.2 BoxCast（盒状射线检测）

[p.21]

BoxCast 不是发射一条线，而是发射一个矩形（3D 中为立方体）沿方向移动，检测途中碰撞到的物体。

// 2D 版本

```
RaycastHit2D BoxCast(Vector2 origin, Vector2 size, float angle,
    Vector2 direction, float distance, int layerMask);
```

参数解析：

- **origin:** 矩形的起始中心位置——设为角色碰撞盒中心
- **size:** 矩形的长宽——设为角色碰撞盒 size（整个角色宽度都在检测，解决平台边缘问题）
- **angle:** 旋转角度——设为 0
- **direction:** 投射方向——设为 `Vector2.Down`
- **distance:** 投射距离——设一个很小的值（只需检测脚下紧邻区域，跳起来时不会检测到地面，也适应缓坡）

7.3 CircleCast / SphereCast

[p.22]

发射一个圆形（2D）或球体（3D）进行检测。

```
RaycastHit2D CircleCast(Vector2 origin, float radius,
    Vector2 direction, float distance, int layerMask);
```

还有 `CapsuleCast`——发射一个胶囊形状。选择原则：**根据具体使用场景选择最合适的检测形状**。人物角色通常用 `CapsuleCast`（类似角色碰撞体形状）、子弹用 `SphereCast`、地面检测用 `BoxCast`。

7.4 All 与 NonAlloc 版本

[p.25 习题 2]

几乎所有射线检测函数都有三种版本：

- **普通版本**（如 `BoxCast`）：返回第一个碰撞到的物体
- **All 版本**（如 `BoxCastAll`）：返回所有碰撞到的物体（分配新数组，有 GC 开销）
- **NonAlloc 版本**（如 `BoxCastNonAlloc`）：**性能最优**。需要预先分配一个数组作为缓冲区，不产生 GC。适合在 `Update` 中频繁调用

8 射线检测可视化

[p.23-24]

使用 `Debug.DrawLine()` 和 `Debug.DrawRay()` 在 `Scene` 视图中可视化射线检测。

基本 Raycast 可视化：

```
void DrawRaycast2D(Vector2 origin, Vector2 direction, float distance) {
    Debug.DrawRay(origin, direction * distance, Color.red, Time.deltaTime);
}
```

BoxCast 可视化：需要绘制起点和终点两个矩形，并用线段连接四个角。核心逻辑：

1. 在 `origin` 处绘制 `size×size` 的矩形（起点框）
2. 在 `origin + direction * distance` 处绘制同样矩形（终点框）
3. 用四条线段连接两个矩形的四个角

旋转时矩形需要额外旋转处理（通过 `RotateVector` 实现）。

CircleCast 可视化：`Debug` 类没有直接画弧线的函数，但可以用 `Gizmos.DrawWireSphere`。更好的做法是在 `OnDrawGizmos` 中使用 `Gizmos` 绘制起点圆和终点圆。

9 课后习题详解

[p.25]

9.1 题 1：搭建场景演示 DCD 和两种 CCD 的区别

实验设计：

```
// 场景设置：
// 1. 一面薄墙（厚度很小，如0.1单位）
// 2. 三个相同的球体，分别设置不同的Collision Detection模式
// 3. 给球体相同的极高速度（如100单位/秒）撞向墙壁
// 预期结果：
// - Discrete球：直接穿过墙壁（隧道效应）
// - Continuous球：正常碰撞反弹
// - Continuous Speculative球：正常碰撞反弹（可能稍有不同）
```

在 Rigidbody 组件中分别设置 `collisionDetection` 为 `Discrete`、`Continuous`、`Continuous Speculative`，观察差异。

9.2 题 2：All 和 NonAlloc 版本的用途

- **BoxCastAll**：返回所有碰到物体的数组。适合需要知道射线穿过的所有物体时（如穿透性子弹伤害所有路径上的敌人）。缺点是每次调用都会分配新数组（产生 GC 垃圾）。
- **BoxCastNonAlloc**：接受一个预分配的 **RaycastHit2D** 数组作为参数，将结果写入数组，返回实际命中的数量。**零 GC 分配**，适合在 `Update`/`FixedUpdate` 中频繁调用。

最佳实践：频繁调用的射线检测（如每帧地面检测）使用 `NonAlloc` 版本，预分配并复用数组；一次性检测（如点击拾取物品）使用普通版本或 `All` 版本即可。

9.3 题 3：使用 Physics.Simulate 预测物体轨迹

`Physics.Simulate()` 允许在指定的 `PhysicsScene` 上手动执行一次物理模拟步进。

预测未来 60 帧轨迹的思路：

```
IEnumerator PredictTrajectory(Rigidbody rb, int frames = 60) {
    // 1. 克隆场景或记录初始状态
    Vector3 startPos = rb.position;
    Quaternion startRot = rb.rotation;
    Vector3 startVel = rb.linearVelocity;
    Vector3 startAngVel = rb.angularVelocity;

    List<Vector3> trajectory = new List<Vector3>();

    // 2. 模拟60步
    for (int i = 0; i < frames; i++) {
```

```
        Physics.Simulate(Time.fixedDeltaTime); // 手动步进一步
        trajectory.Add(rb.position);
    }

    // 3. 恢复初始状态
    rb.position = startPos;
    rb.rotation = startRot;
    rb.linearVelocity = startVel;
    rb.angularVelocity = startAngVel;

    // 4. 可视化轨迹
    for (int i = 0; i < trajectory.Count - 1; i++) {
        Debug.DrawLine(trajectory[i], trajectory[i+1], Color.green, 5f);
    }
}
```

关键点：模拟完成后必须恢复物体的原始状态（位置、旋转、速度），因为 `Physics.Simulate` 会真实地修改物理世界状态。